LAB

7

Operating Systems · FCIM / FAF · UTM · 2026-2027

Three Small Programs: Processes, Redirection, a Pipe

A **Track A: the real OS, from the outside (Linux) · 90 minutes**

AT A GLANCE

Part 1	fork + exec + wait	~25 min
Part 2	A zombie, on purpose	~15 min
Part 3	Redirection: ls -l > out.txt	~20 min
Part 4	A pipe: ls /etc | wc -l	~25 min
You submit	run.c, zombie.c, redirect.c, pipeline.c (and pipeline3.c if you did it).	

Goal

The lecture asked three questions: how is a process born, where does printf write, and how does ls | wc -l work. Today you run the answers. You get three short C programs, already written: you compile them, run them, change one line and look at what happens.

Before you start

- Work on your own Linux laptop. gcc is already there if you ran os2026-setup.sh.
- Keep a file report.md open: every "Observe" box asks for one or two short answers there.

TERMINAL

```
mkdir -p ~/os-lab7 && cd ~/os-lab7
gcc --version | head -1
```

Minimum to pass

Parts 1 and 2, with their answers in report.md. Parts 3 and 4 bring the rest of the grade. The optional part is not graded.

PART 1 fork + exec + wait ~25 min

This program does what the shell does when you type ls -l: it copies itself (fork), the copy becomes ls (exec), and the original waits for it (wait).

1.1 Create run.c with exactly this content:

FILE run.c

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>

/* the three steps of the shell for the command:   ls -l   */
int main(void)
```

```
{
    printf("parent: my PID is %d\n", getpid());
    fflush(stdout);

    pid_t pid = fork();                          /* step 1: copy myself */
    if (pid < 0) { perror("fork"); return 1; }

    if (pid == 0) {                              /* only the child runs this */
        printf("child:  my PID is %d, my parent is %d\n", getpid(), getppid());
        fflush(stdout);
        execlp("ls", "ls", "-l", (char *)NULL);  /* step 2: become ls */
        perror("exec");                          /* runs only if exec failed */
        exit(127);
    }

    int status;
    waitpid(pid, &status, 0);                    /* step 3: wait for the child */
    printf("parent: child %d ended with status %d\n", pid, WEXITSTATUS(status));
    return 0;
}
```

1.2 Compile and run it:

TERMINAL

```
gcc -Wall -Wextra -o run run.c && ./run
```

Expected output:

EXPECTED OUTPUT

```
parent: my PID is 15
child:  my PID is 16, my parent is 15
total 20
-rwxr-xr-x 1 root root 16136 Oct 18 20:25 run
-rwxr-xr-x 1 root root   911 Oct 18 20:25 run.c
parent: child 16 ended with status 0
```

(Your PIDs and your listing will be different. The order of the lines is what matters.)

1.3 Make the child run another program. Replace the execlp line with the one below, compile and run again:

CODE

```
        execlp("date", "date", (char *)NULL);
```

1.4 Now ask for a program that does not exist, compile and run:

CODE

```
        execlp("nosuchprog", "nosuchprog", (char *)NULL);
```

Expected output (the end):

EXPECTED OUTPUT

```
exec: No such file or directory
parent: child 32 ended with status 127
```

Observe (write in report.md)

1. The line perror("exec") is in the child, right after execlp. Why is it printed only with nosuchprog?
2. Which number does the child print as "my parent", and whose PID is it?
3. What status does the parent see with ls, and what status with nosuchprog?

PART 2 A zombie, on purpose ~15 min

A zombie is a child that has ended but has not been waited for. You make one by letting the parent sleep before wait.

2.1 Copy the original run.c (the version with ls) to zombie.c and add one line, sleep(20);, just before waitpid:

FILE zombie.c

```
    int status;
    sleep(20);                                  /* the new line */
    waitpid(pid, &status, 0);                   /* step 3: wait for the child */
```

2.2 Compile it, start it in the background and look at the process table while the parent sleeps:

TERMINAL

```
gcc -Wall -Wextra -o zombie zombie.c
./zombie > /dev/null &
sleep 2; ps -o pid,ppid,stat,cmd
```

Expected output:

EXPECTED OUTPUT

```
  PID  PPID STAT CMD
   39     1 S    ./zombie
   41    39 Z    [ls] <defunct>
   42     1 R    ps -o pid,ppid,stat,cmd
```

(Your PIDs will be different, and you will also see the line of your own shell.)

2.3 Wait 20 seconds and run `ps -o pid,ppid,stat,cmd` again.

Observe (write in report.md)

1. Which letter does ls have in the STAT column, and what does it mean?
2. After 20 seconds, is the [ls] <defunct> line still there? Which line of zombie.c made it disappear?

PART 3 Redirection: ls -l > out.txt ~20 min

This program does what the shell does for ls -l > out.txt: in the child it closes 1, opens the file (which gets the smallest free number, 1), then runs ls.

3.1 Create redirect.c:

FILE redirect.c

```
#include <stdio.h>
#include <stdlib.h>
```

```c
#include <fcntl.h>
#include <unistd.h>
#include <sys/wait.h>

/* does the same as the shell command:   ls -l > out.txt   */
int main(void)
{
    pid_t pid = fork();
    if (pid == 0) {
        close(1);                                /* descriptor 1 is now free */
        int fd = open("out.txt", O_WRONLY | O_CREAT | O_TRUNC, 0644);
        if (fd < 0) { perror("open"); exit(1); }
        fprintf(stderr, "open gave descriptor %d\n", fd);
        execlp("ls", "ls", "-l", (char *)NULL);  /* ls writes to 1, as always */
        perror("exec");
        exit(127);
    }
    waitpid(pid, NULL, 0);
    printf("done: the listing is in out.txt\n");
    return 0;
}
```

3.2 Compile, run, then look inside the file:

TERMINAL

```
gcc -Wall -Wextra -o redirect redirect.c && ./redirect
head -3 out.txt
```

Expected output:

EXPECTED OUTPUT

```
open gave descriptor 1
done: the listing is in out.txt
total 100
-rw-r--r-- 1 root root     0 Oct 18 20:25 out.txt
-rwxr-xr-x 1 root root 16136 Oct 18 20:25 redirect
```

Observe (write in report.md)

1. Which number did open give, and why exactly that one?
2. ls was not changed at all. Why does its listing not appear on the screen?
3. To redirect the errors instead (like 2> err.txt), which descriptor would you close before open?

PART 4 A pipe: ls /etc | wc -l ~25 min

This program builds the pipeline by hand: one pipe, two forks. In ls, descriptor 1 becomes the writing end p[1]; in wc, descriptor 0 becomes the reading end p[0]. dup copies a descriptor into the smallest free number, exactly like open.

4.1 Create pipeline.c:

FILE pipeline.c

```c
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
```

```
#include <sys/wait.h>

/* does the same as the shell command:   ls /etc | wc -l   */
int main(void)
{
    int p[2];
    if (pipe(p) < 0) { perror("pipe"); return 1; }

    if (fork() == 0) {                          /* ls: its 1 becomes p[1] */
        close(1);
        dup(p[1]);                              /* smallest free number: 1 */
        close(p[0]);
        close(p[1]);
        execlp("ls", "ls", "/etc", (char *)NULL);
        perror("exec ls");
        exit(127);
    }

    if (fork() == 0) {                          /* wc: its 0 becomes p[0] */
        close(0);
        dup(p[0]);                              /* smallest free number: 0 */
        close(p[0]);
        close(p[1]);
        execlp("wc", "wc", "-l", (char *)NULL);
        perror("exec wc");
        exit(127);
    }

    close(p[0]);                                /* the parent uses neither end */
    close(p[1]);
    wait(NULL);
    wait(NULL);
    return 0;
}
```

4.2 Compile, run, and compare with the real shell. The two numbers must be equal:

TERMINAL

```
gcc -Wall -Wextra -o pipeline pipeline.c && ./pipeline
ls /etc | wc -l
```

4.3 Break it on purpose. In the parent, turn the line close(p[1]); (the one right after the comment "the parent uses neither end") into a comment, compile and run:

CODE

```
    close(p[0]);                                /* the parent uses neither end */
    /* close(p[1]); */
```

The program does not end. Stop it with Ctrl-C.

Observe (write in report.md)

1. Did ./pipeline and ls /etc | wc -l give the same number?
2. After the change, which of the two children never ends, ls or wc?
3. Why, in one sentence: what is wc waiting for?

Optional, not graded: three stages

Write pipeline3.c for `ls /usr/bin | grep x | wc -l`: two pipes, three children. Every child closes all four pipe ends after its dup, and so does the parent. Check that it prints the same number as the shell.

FINAL Wrap-up and deliverable ~5 min

You submit:

- ☐ run.c, zombie.c, redirect.c, pipeline.c (and pipeline3.c if you did it).
- ☐ report.md: the answers to the "Observe" boxes.

Push everything to your course repository; the assistant pulls it and compiles it.

Lab report (required):

- Every lab must be accompanied by a written report: what you did, the answers to all "Observe" boxes with the commands and outputs (or screenshots) that support them, and a short conclusion.
- Labs 7 and 8 may be combined into a single report.
- Everything you submit (the report and any code) must also be pushed to your Git repository.